

Lab 06: A “gentle” Introduction to Artificial Neural Networks in R

Andrés Oswaldo Calderón Romero, PhD.

May 7, 2026

1 Core Concepts of ANN

An Artificial Neural Network (ANN) is a set of connected input-output units where each connection has an associated weight. In the learning phase, the network adjusts these weights to predict the correct target values of input tuples.

A standard unit is a mathematical function that takes a linear weighted sum of inputs and passes it through a nonlinear activation function, such as the sigmoid function. The net input is defined as $I = \sum_{i=1}^n X_i w_i + b$, and the final output is $O = f(I)$ (Fritsch and Guenther 2010).

2 Environment Setup

To begin, we utilize the “**neuralnet**” package (Fritsch, Guenther, and Wright 2019), which is designed for training multilayer feed-forward networks.

```
# Install and load the library
if(!require(neuralnet)) install.packages("neuralnet")
library(neuralnet)
```

Listing 1: Environment setup and library initialization. This step involves installing and loading the “**neuralnet**” package required for training feed-forward networks.

3 Data Preprocessing

Neural networks often require data to be scaled, as functions like the sigmoid (squashing) function map inputs to a range of 0 to 1.

```

# Using the built-in iris dataset
data(iris)

# Binary classification: Is the species "setosa"?
iris$is_setosa <- ifelse(iris$Species == "setosa", 1, 0)

# Normalization to [0, 1] range
maxs <- apply(iris[, 1:4], 2, max)
mins <- apply(iris[, 1:4], 2, min)
scaled <- as.data.frame(scale(iris[, 1:4], center = mins, scale = maxs - mins))
final_data <- cbind(scaled, is_setosa = iris$is_setosa)

# Splitting into Training (70%) and Test (30%)
set.seed(42)
index <- sample(1:nrow(final_data), 0.7 * nrow(final_data))
train_set <- final_data[index, ]
test_set <- final_data[-index, ]

```

Listing 2: Data preprocessing and partitioning. This listing covers loading the Iris dataset, performing range normalization (scaling inputs from 0 to 1), and splitting the data into training (70%) and testing (30%) subsets.

4 Training the Model

We train the network using the backpropagation algorithm, which iteratively modifies weights in a “backward” direction to minimize the error between predictions and actual targets.

```

# Training a network with one hidden layer of 3 units
# This is a two-layer neural network (input layer not counted)
nn <- neuralnet(is_setosa ~ Sepal.Length + Sepal.Width + Petal.Length + Petal.Width,
               data = train_set,
               hidden = 3,
               linear.output = FALSE)

```

Listing 3: Training the multilayer feed-forward neural network. The code demonstrates training a model with one hidden layer of three units using the backpropagation algorithm to minimize prediction error.

5 Visualization

The architecture of a multilayer feed-forward network consists of an input layer, one or more hidden layers, and an output layer.

```
# Plotting the network architecture
plot(nn)
```

Listing 4: Visualization of the network architecture. This command generates a visual representation of the input, hidden, and output layers of the trained model.

6 Evaluation

To classify unknown tuples, we input them into the trained network and compute the output units' values. For binary tasks, a value ≥ 0.5 is typically considered the positive class.

```
# Prediction on test data
results <- compute(nn, test_set[, 1:4])
predictions <- ifelse(results$net.result > 0.5, 1, 0)

# Confusion Matrix and Accuracy
conf_matrix <- table(Actual = test_set$is_setosa, Predicted = predictions)
accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix)

print(conf_matrix)
print(paste("Test Accuracy: ", round(accuracy, 4)))
```

Listing 5: Model evaluation and performance metrics. This section calculates predictions on test data, generates a confusion matrix, and computes the final test accuracy.

7 Predicting New Tuples

We can now predict the class of a new observation. Note that new data must undergo the same normalization as the training data.

```
# New tuple: SL=5.0, SW=3.4, PL=1.5, PW=0.2
new_tuple <- data.frame(Sepal.Length=5.0, Sepal.Width=3.4, Petal.Length=1.5, Petal.Width=0.2)
new_scaled <- as.data.frame(scale(new_tuple, center = mins, scale = maxs - mins))

# Compute prediction
pred_new <- compute(nn, new_scaled)
print(paste("Probability of Setosa: ", round(pred_new$net.result, 4)))
```

Listing 6: Predictive inference for a new data observation. This final step scales a new data tuple and uses the trained network to compute the probability of it belonging to a specific class.

References

- Fritsch, Stefan and Frauke Guenther (2010). “neuralnet: Training of Neural Networks”. In: *The R Journal* 2.1, pp. 30–38. DOI: 10.32614/RJ-2010-011.
- Fritsch, Stefan, Frauke Guenther, and Marvin N. Wright (2019). *neuralnet: Training of Neural Networks*. R package version 1.44.2. URL: <https://CRAN.R-project.org/package=neuralnet>.